Universidade Cruzeiro do Sul

TÓPICOS AVANÇADOS EM SISTEMAS DE INFORMAÇÃO II

Prof. José Claudio de Sousa
jcsousa@cruzeirodosul.edu.br

Universidade Cruzeiro do Sul

ARQUITETURA DE DESENVOLVIMENTO DE SISTEMAS

BOAS PRÁTICAS DE ARQUITETURA DE SOFTWARE

BOAS PRÁTICAS ARQUITETURA DE SOFTWARE

As boas práticas na Arquitetura de Software ajudam a garantir que o sistema seja escalável, robusto e fácil de manter a longo prazo.
Em seguida falaremos de algumas dessas boas práticas.

COMPREENSÃO DOS REQUISITOS NÃO FUNCIONAIS

Os requisitos não funcionais (performance, segurança, escalabilidade, etc.) são tão importantes quanto os funcionais. Uma boa arquitetura deve considerar esses requisitos desde o início, pois eles influenciam diretamente o design do sistema. Por exemplo, um sistema que precisa ser altamente escalável deve evitar um design que dependa de recursos centralizados ou gargalos.

MODULARIDADE E SEPARAÇÃO DE RESPONSABILIDADES

Modularidade é essencial para manter a coesão do sistema e diminuir o acoplamento. Cada módulo deve ter uma responsabilidade bem definida e interagir com outros módulos por meio de interfaces claras. Essa separação torna o sistema mais fácil de testar, manter e expandir.

Módulo de Usuários: Responsável por registro, login, perfil.

Interface (API): POST /auth/login, GET /users/profile, PUT /users/profile

Módulo de Produtos: Responsável por catálogo, busca, detalhes.

Interface (API): GET /products, GET /products/{id}, GET /products?search=termo

Módulo de Pedidos: Responsável por carrinho, checkout, histórico.

Interface (API): POST /orders, GET /orders/{id}, GET /orders/user/{userId}

USO DE PADRÕES ARQUITETURAIS

Adotar **padrões de projeto arquiteturais** reconhecidos é uma maneira eficaz de resolver problemas recorrentes. Exemplos incluem:

MVC (Model-View-Controller) para aplicações web.

Event-Driven Architecture (Arquitetura Orientada a Eventos) para sistemas reativos.

Microserviços para sistemas distribuídos.

CQRS (Command Query Responsibility Segregation) para separar leituras de gravações em sistemas complexos.

Esses padrões fornecem uma estrutura comprovada para o desenvolvimento, economizando tempo e evitando problemas futuros

ENCAPSULAMENTO E ABSTRAÇÃO

A prática de encapsulamento e abstração ajuda a ocultar os detalhes de implementação dos módulos e a expor apenas as interfaces necessárias. Isso melhora a **manutenibilidade** e **substituibilidade** dos componentes, permitindo atualizações sem afetar o restante do sistema.

BAIXO ACOPLAMENTO E COESÃO

Baixo acoplamento significa que os componentes do sistema devem ter dependências mínimas entre si. Alta coesão garante que as funcionalidades de um módulo estejam fortemente relacionadas, o que facilita a leitura, manutenção e evolução do código. Sistemas com baixo acoplamento e alta coesão tendem a ser mais flexíveis e fáceis de adaptar a novas demandas.

DOCUMENTAÇÃO ARQUITETURAL

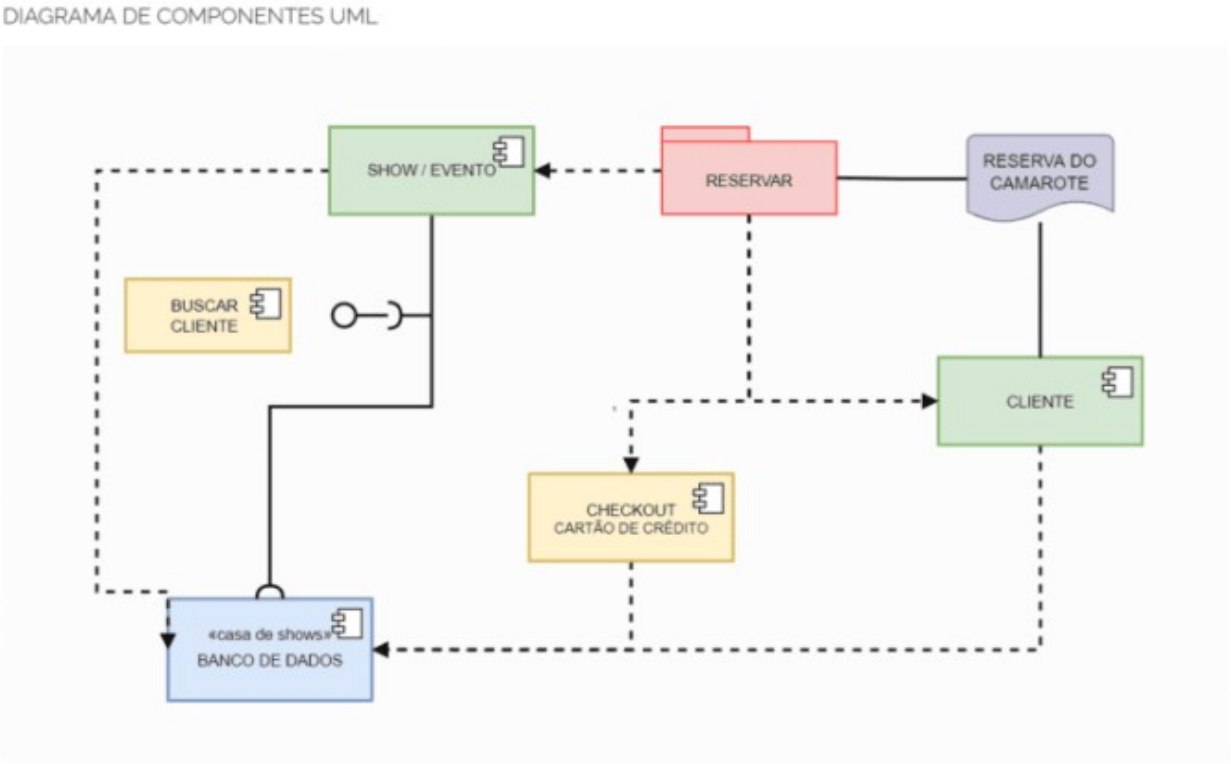
Uma boa documentação é essencial para a continuidade do projeto e a transmissão do conhecimento arquitetural entre equipes. Documentar os principais componentes, suas interações e as decisões de design ajudam a garantir que o sistema seja compreendido por outros desenvolvedores e arquitetos, mesmo após anos de uso. Exemplos de documentações importantes incluem:

- Diagramas de componentes.
- Diagramas de sequência.
- Casos de uso.
- Diagramas de atividades e fluxos de dados.

DOCUMENTAÇÃO ARQUITETURAL

Diagramas de componentes

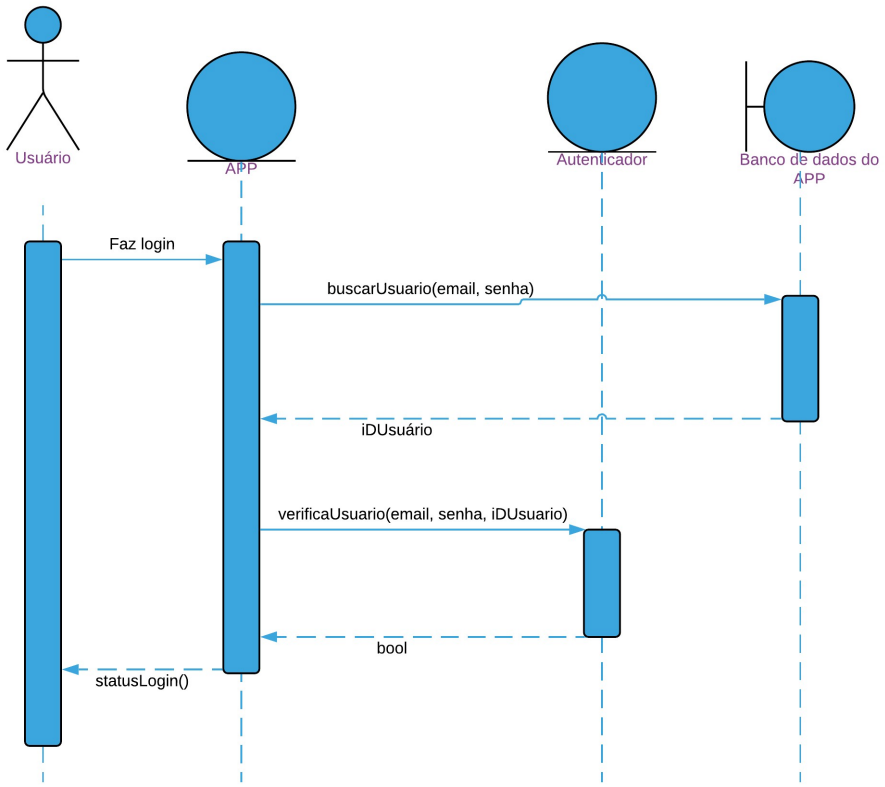
Mostra a estrutura física do sistema, como os componentes de software (módulos, bibliotecas, serviços) estão organizados e como se relacionam através de interfaces bem definidas. Foca nas **dependências entre partes do sistema**.



DOCUMENTAÇÃO ARQUITETURAL

Diagramas de sequência

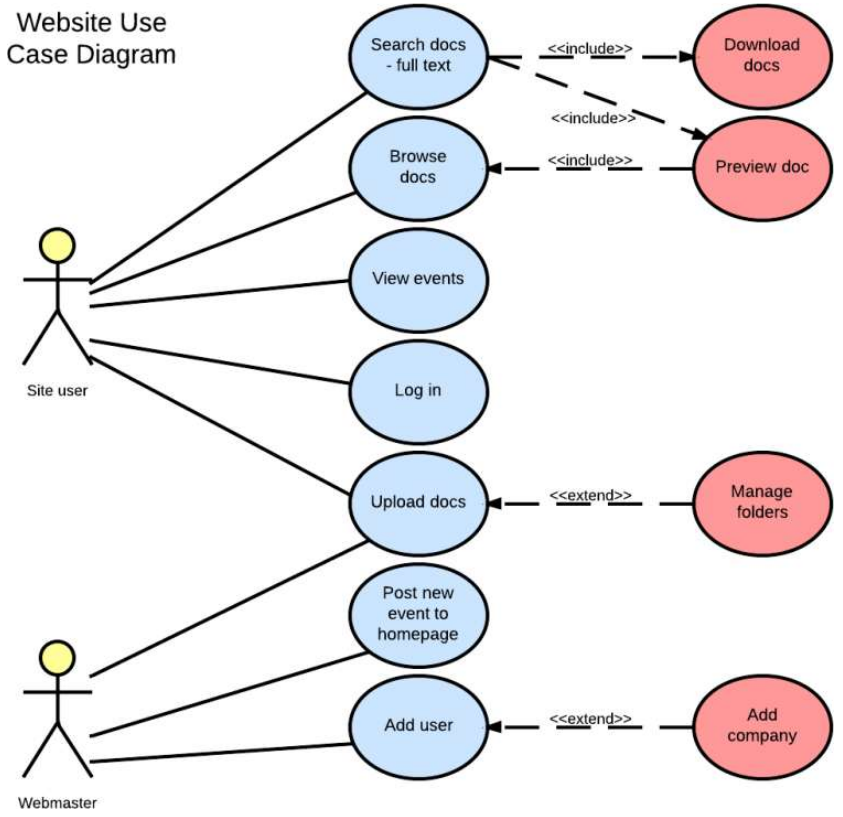
Modela a **interação entre objetos/componentes ao longo do tempo**, mostrando a ordem temporal das mensagens trocadas. É ideal para detalhar a lógica de um caso de uso específico.



DOCUMENTAÇÃO ARQUITETURAL

Casos de uso

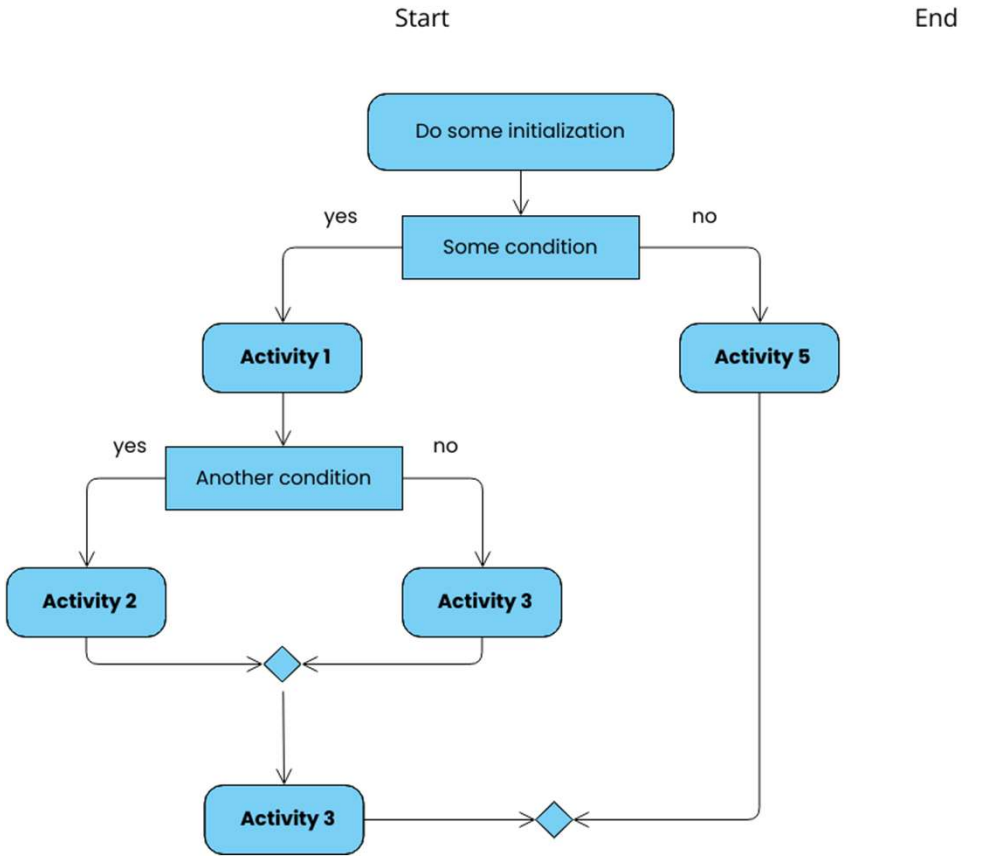
Descreve a **interação entre um ator (usuário ou sistema externo) e o sistema** para atingir um objetivo específico. Foca na **funcionalidade** do ponto de vista do usuário, não na implementação.



DOCUMENTAÇÃO ARQUITETURAL

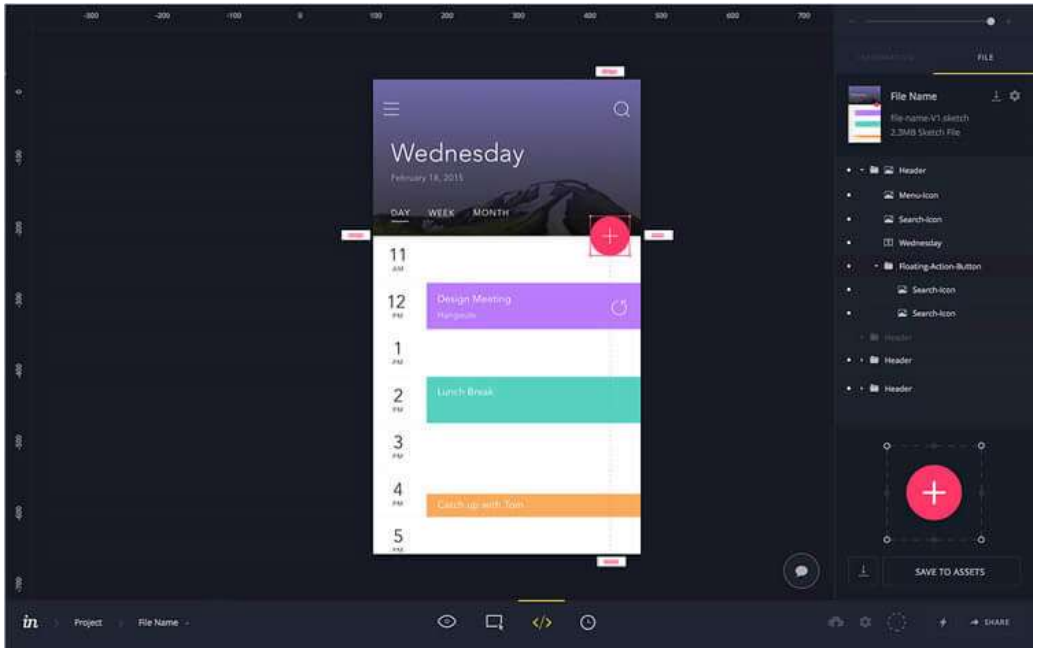
Diagramas de atividades e fluxos de dados.

Modela a **lógica de um processo de negócio ou fluxo de trabalho**, mostrando o fluxo de controle de uma atividade para outra. É como um fluxograma avançado.



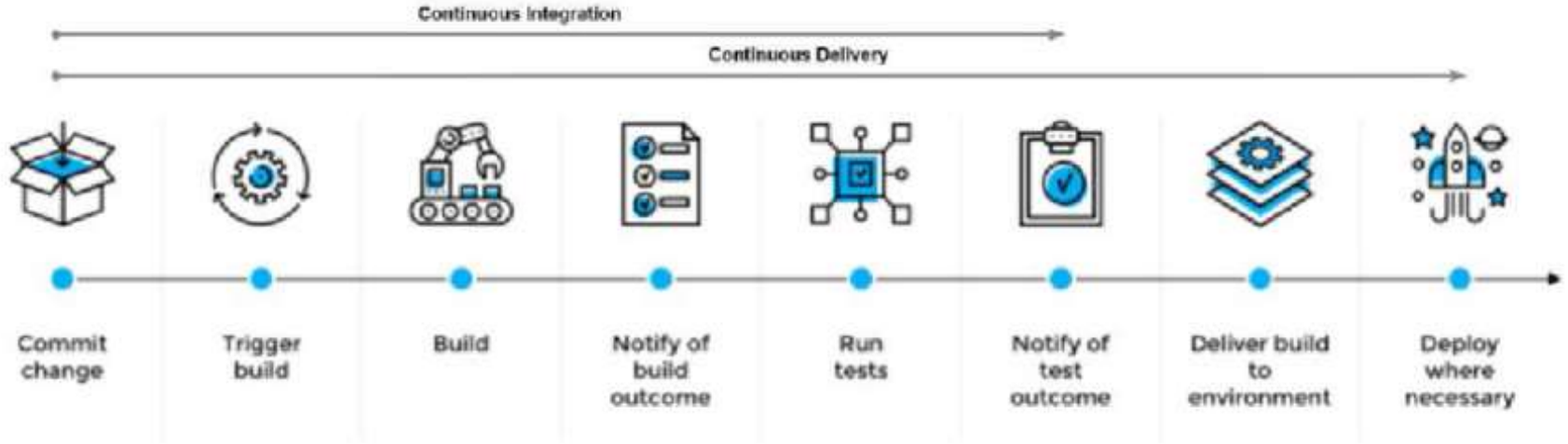
VALIDAÇÃO ARQUITETURAL

Antes de seguir adiante com a implementação, é importante validar as escolhas arquiteturais. Ferramentas de **prototipagem** e **simulações** podem ser usadas para testar a escalabilidade e o desempenho do design. Além disso, revisões arquiteturais periódicas, envolvendo tanto os desenvolvedores quanto stakeholders, garantem que a arquitetura esteja alinhada com os requisitos do negócio.



AUTOMAÇÃO E INTEGRAÇÃO CONTÍNUA

Utilizar **pipelines de integração contínua (CI)** e **entrega contínua (CD)** permite que novas funcionalidades e correções sejam integradas rapidamente ao sistema, minimizando o risco de defeitos. A automação de testes e deploys não só agiliza o desenvolvimento, mas também garante a qualidade e a consistência do sistema.



MONITORAMENTE E OBSERVABILIDADE

A **observabilidade** de um sistema em produção é fundamental para garantir sua saúde e desempenho ao longo do tempo. Adicionar métricas, logs e rastreamento de eventos permite que a equipe detecte problemas com rapidez e aja preventivamente antes que eles se tornem críticos. Ferramentas como **Prometheus**, **Grafana**, e **Elasticsearch** são frequentemente usadas para este fim.

Universidade Cruzeiro do Sul

MONITORAMENTE E OBSERVABILIDADE





Dashboard Grafana showcase

Universidade Cruzeiro do Sul

DESIGN PARA MUNDAÇAS

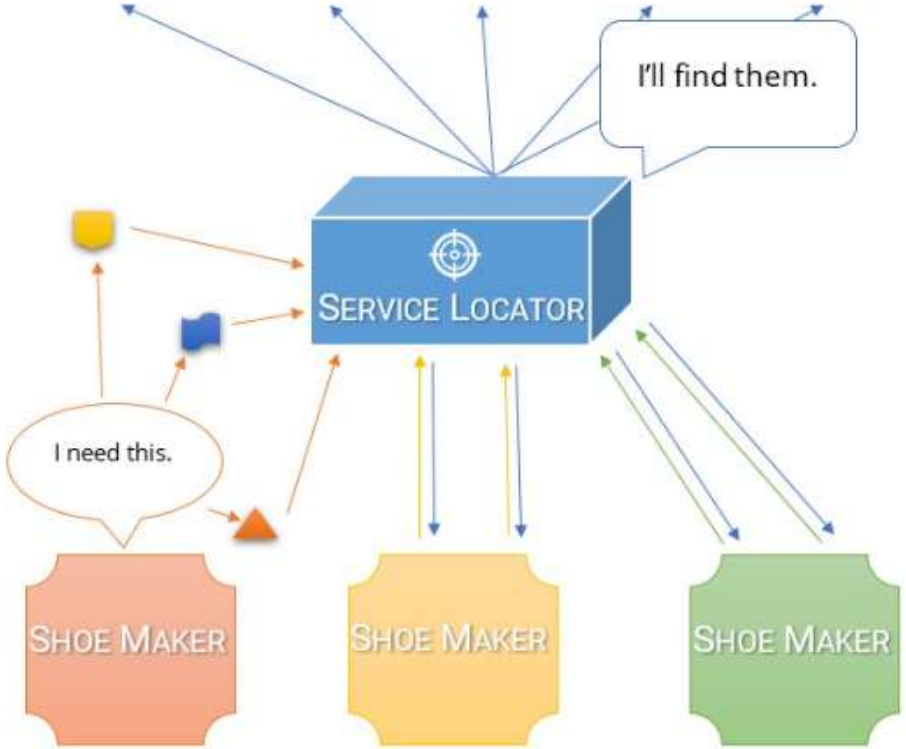
Sistemas de software precisam estar prontos para se adaptar a mudanças nos requisitos ao longo do tempo. Assegure que sua arquitetura possa ser **escalada** ou **modificada** com o mínimo de impacto possível. Para isso, use práticas como o design orientado a interfaces, modularização e padrões como **Dependency Injection** e **Service Locator**.

DESIGN PARA MUNDAÇAS

Dependency Injection



Service Locator

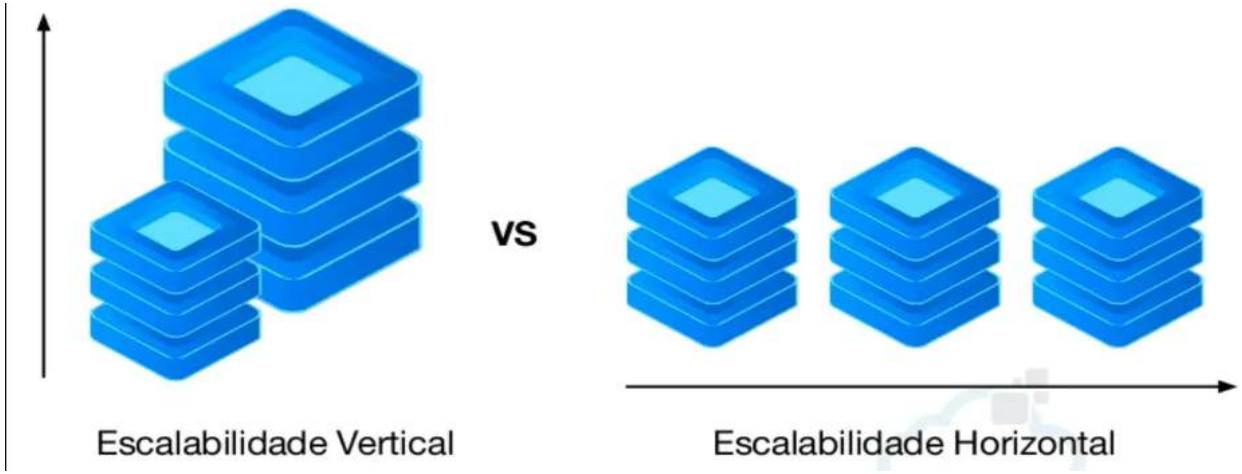


SEGURANÇA DESDE O INÍCIO

Incorporar **práticas de segurança** desde o início do projeto é fundamental. Utilize autenticação robusta, criptografia para dados sensíveis e implemente controles de acesso adequados. Além disso, realize auditorias de segurança e testes de penetração para garantir que o sistema esteja seguro contra ameaças externas.

ESCALABILIDADE HORIZONTAL

Sempre que possível, prefira **escalabilidade horizontal** (adicionar mais máquinas/instâncias) à **escalabilidade vertical** (melhorar o hardware de uma máquina). Isso oferece maior flexibilidade e resistência a falhas. Arquiteturas baseadas em microserviços são especialmente boas para essa abordagem, pois permitem escalar apenas os componentes que precisam de mais recursos.



REUSO DE COMPONENTES

Se possível, reutilize componentes, bibliotecas ou frameworks que já tenham sido testados e comprovados. Isso não só acelera o desenvolvimento, mas também reduz o risco de introdução de novos erros. O uso de **APIs externas** e **componentes de terceiros** também pode ser explorado para tarefas que não sejam o foco principal do projeto.

DESENVOLVIMENTO BASEADO EM DOMÍNIO (DDD - DOMAIN-DRIVEN DESIGN)

O **DDD** ajuda a criar uma estrutura arquitetural alinhada ao domínio do problema, assegurando que o software reflita corretamente as necessidades e complexidades do negócio. Essa prática facilita a comunicação entre técnicos e não-técnicos e organiza melhor as funcionalidades dentro do sistema.

Universidade Cruzeiro do Sul

TOLERÂNCIA A FALHAS

Desenvolva seu sistema com a capacidade de resistir e se recuperar de falhas. Implementar técnicas como **circuit breakers**, **retry logic**, e **fallbacks** garante que, mesmo diante de erros ou quedas de serviço, o sistema pode continuar a operar ou se recuperar rapidamente.

A IMPORTÂNCIA DAS BOAS PRÁTICAS NA ARQUITETURA DE SOFTWARE

Como vimos, a qualidade arquitetural de um software é construída desde o primeiro dia através de **decisões conscientes e boas práticas**.

A combinação de **requisitos bem definidos**, **modularidade eficaz**, **padrões arquiteturais apropriados** e princípios como **baixo acoplamento e alta coesão** formam o alicerce para sistemas manteníveis e escaláveis.

Quando documentamos essas escolhas arquiteturais, não apenas registramos decisões, mas criamos um guia para evolução futura do sistema. Essas práticas, aplicadas consistentemente, são o que separa projetos sustentáveis daqueles que acumulam dívida técnica rapidamente.

Universidade Cruzeiro do Sul

